Length, Search, Count and Replace operations for Linked Lists

Kevin Browne

Linked list operations

- There are **many** operations/algorithms we can implement with linked lists
- Think of all the types of algorithms that use arrays!
- Certain base operations like length, search, delete matches are typical
  - But we can also get into more advanced operations, like sorting or merging linked lists too!

Length

- To compute a linked list's length, we can traverse the list and keep track of how many nodes with a counter variable
- To an extent our print function already does this
- We can implement a length function in the same way, and have it return the final computed length

length function

```
int length(Node *head)
{
  Node *current;
  current = head;
  int length = 0;
  while (current != NULL)
  {
    length++;
    current = current->next;
  }
  return length;
}
```

Recursive functions

- Often times it's easier to write **recursive** functions to deal with data structures
- NULL/empty can be treated as a **base case** that results in a trivial value
- Dealing with other nodes becomes a **recursive step** that results in a recursive function call (possibly with a value too)
- We can write a fairly elegant looking recursive length function this way...

Isn't this nice to look at and comprehend?

```
int recursive_length(Node *node)
{
  if (node == NULL) return 0;
  else return 1 + recursive_length(node->next);
}
```

A lot of the time recursion can just make things more complicated to understand... but is that really the case here?

Length functions will both produce the same result

```
Node 0: 5
Node 1: 7
Node 2: 10


length: 3
recursive length: 3
```

Searching a linked list

- We might want to search a linked list in all kinds of different ways...
  - Check if a node matches a given value
  - Count how many nodes match a given value
  - Determine the position or positions of matching nodes

- Let's check if a node matches a given value in a linked list using recursion again
  - For NULL/empty we would return false (no match)
  - If the node contains the matching value we can return true, otherwise we just check the next node in the list!

Recursive membership check

```
bool is_member(Node *node, int find_value)
{
  if (node == NULL) return false;
  else if (node->value == find_value) return true;
  else return is_member(node->next, find_value);
}
```

Checking for members…

```
Node 0: 5
Node 1: 7
Node 2: 10

3 is not a member of the list
7 is a member of the list
```

Count matches

- Counting the number of nodes with a matching value is a very similar algorithm
- Our base case NULL/empty is a result of 0
- And our recursive step can check the value of the current node
  - If it matches, the result is 1 + the match count of the remaining nodes
  - If it does not match, the result is the match count of the remaining nodes

Count matches function

```
int count_matches(Node *node, int find_value)
{
  if (node == NULL) return 0;
  else if (node->value == find_value)
    return 1 + count_matches(node->next,find_value);
  else return count_matches(node->next, find_value);
}
```

Counting matches…

```
Node 0: 5
Node 1: 7
Node 2: 10
Node 3: 7
Node 4: 7
Node 5: 10

number of 7s: 3
number of 10s: 2
number of 5s: 1
```

Replace matches

- We could similarly write a function to replace matching values in a linked list with another value
- We could recursively traverse through the linked list
- In the case of NULL/empty, we don’t do anything!
- In the case of a regular node, we could replace the value if it matches the value we are replacing
  - And whether it matches or not, we would always continue to check the next node

Replace matches function

```
void replace_matches(Node *node, int find_value, int replace_value)
{
  if (node != NULL) {
    if (node->value == find_value) node->value = replace_value;
    replace_matches(node->next, find_value, replace_value);
  }
}
```

Replacing matches...

```
List before replacement...
Node 0: 5
Node 1: 7
Node 2: 10
Node 3: 7
Node 4: 7
Node 5: 10

List after replacing 10 with 14....
Node 0: 5
Node 1: 7
Node 2: 14
Node 3: 7
Node 4: 7
Node 5: 14
```